CO3 – Individual Assignment – Q30

1. HEADER

Name: Sasi Kumar.S
Roll No.:192324254
Course Code & CO Reference: CSA0920-CO3
Assignment Set: Direct-Descriptive
Question No.: Q30

Assigned Question

Q30. Race Condition Demonstration

Write a Java program that creates a shared counter variable and starts multiple threads that each increment this counter a large number of times without any synchronization. Run the program multiple times and observe that the final counter value is often incorrect (less than the expected total), demonstrating a race condition caused by concurrent unsynchronized access to shared data.

Concepts: Race conditions, shared mutable state.

GitHub Repository Link: https://github.com/SasiKumarS254/CSA0920-Programming-in-JAVA/blob/main/30_CO3_Q30

Submission Date: 11/8/2026

2. PROBLEM UNDERSTANDING

The program uses one counter that is shared by multiple threads. Each thread increases the same counter many times without synchronization. Since the threads can access the counter at the same time, some increments may be missed and the final value can be less than the expected value.

3. APPROACH / DESIGN NOTES

- Created a `Counter` class to store the shared counter value.
- Created a `CounterThread` class by extending `Thread`.
- Used `Scanner` to get the number of threads and number of increments from the user.
- All threads use the same `Counter` object.
- Used `join()` so that the main thread waits until all threads complete.
- No synchronization is used because the purpose of the program is to demonstrate a race condition.

- The expected value is calculated as the number of threads multiplied by the number of increments.

4. SOURCE CODE

```
import java.util.Scanner;

class Counter {

    private int count = 0;

    public void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}

class CounterThread extends Thread {

    private Counter counter;
    private int iterations;

    public CounterThread(Counter counter, int iterations) {
        this.counter = counter;
        this.iterations = iterations;
    }

    public void run() {
        for (int i = 0; i < iterations; i++) {
            counter.increment();
        }
    }
}

public class RaceConditionDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of threads: ");
        int numberOfThreads = sc.nextInt();

        System.out.print("Enter number of increments per thread: ");
        int iterations = sc.nextInt();

        if (numberOfThreads <= 0 || iterations <= 0) {
            System.out.println("Enter positive values.");
            sc.close();
            return;
        }
```

```
        Counter counter = new Counter();

        CounterThread[] threads = new CounterThread[numberOfThreads];

        for (int i = 0; i < numberOfThreads; i++) {
            threads[i] = new CounterThread(counter, iterations);
        }

        for (int i = 0; i < numberOfThreads; i++) {
            threads[i].start();
        }

        try {
            for (int i = 0; i < numberOfThreads; i++) {
                threads[i].join();
            }
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted.");
        }

        int expectedValue = numberOfThreads * iterations;
        int actualValue = counter.getCount();

        System.out.println();
        System.out.println("Expected Counter Value : " + expectedValue);
        System.out.println("Actual Counter Value   : " + actualValue);

        if (actualValue < expectedValue) {
            System.out.println("Race Condition Observed!");
        } else {
            System.out.println("Expected value reached.");
        }

        sc.close();
    }
}
```

5. TEST CASES

#	Input	Expected Output	Actual Output
1 –Typical Case	Threads=4 Increments=100000	Expected Counter Value =400000 Actual value should be less than 400000 in a race-condition run	Expected: 400000 Actual: 108338 Race Condition Observed!
2 – Edge Case	Threads=1 Increments=100000	Expected Counter Value =100000 Actual Counter Value = 100000	Expected: 100000 Actual:100000 Expected value reached.
3 – Boundary Case	Threads=2 Increments=100000	Expected Counter Value =200000 Actual value may be less than 200000	Expected: 200000 Actual:111668 Race Condition Observed!

6. SCREENSHOTS OF EXECUTION

Test Case 1 – Typical Case

Input:

Enter number of threads: 4
Enter number of increments per thread: 100000

Sample Output:

Expected Counter Value : 400000
Actual Counter Value : 108338
Race Condition Observed!

```
E:\SasiJava>javac RaceConditionDemo.java

E:\SasiJava>java RaceConditionDemo
Enter number of threads: 4
Enter number of increments per thread: 100000

Expected Counter Value : 400000
Actual Counter Value   : 108338
Race Condition Observed!
```

Test Case 2 – Edge Case

Input:

Enter number of threads: 1
Enter number of increments per thread: 100000

Output:

Expected Counter Value : 100000
Actual Counter Value : 100000
Expected value reached.

```
C:\Users\DELL>cd /d E:\SasiJava

E:\SasiJava>javac RaceConditionDemo.java

E:\SasiJava>java RaceConditionDemo
Enter number of threads: 1
Enter number of increments per thread: 100000

Expected Counter Value : 100000
Actual Counter Value   : 100000
Expected value reached.
```

Test Case 3 – Boundary Case

Input:

Enter number of threads: 2
Enter number of increments per thread: 100000

Sample Output:

Expected Counter Value : 200000
Actual Counter Value : 111668
Race Condition Observed!

```
E:\SasiJava>javac RaceConditionDemo.java

E:\SasiJava>java RaceConditionDemo
Enter number of threads: 2
Enter number of increments per thread: 100000

Expected Counter Value : 200000
Actual Counter Value   : 111668
Race Condition Observed!
```

7. REFLECTION

I understood how multiple threads can access the same variable at the same time and cause a race condition. I also learned that the final value can change between executions when synchronization is not used.

8. DECLARATION

I declare that this is my own work. Any external or AI assistance used was only for understanding the concept and preparing the program. I tested the program and verified the output before submission.